want to breed agents that can last longer and longer in CartPole (therefore achieving a high score).

We need to do this for all the agents in the population.

Listing 6.11. Evaluate all the agents in the population

```
def evaluate_population(pop):
    tot_fit = 0                          1
    lp = len(pop)
    for agent in pop:                    2
        score = test_model(agent)        3
        agent['fitness'] = score         4
        tot_fit += score
    avg_fit = tot_fit / lp
    return pop, avg_fit
```

- ***1*** Total fitness for this population; used to later calculate the average fitness of the population
- ***2*** Iterates through each agent in the population
- ***3*** Runs the agent in the environment to assess its fitness
- ***4*** Stores the fitness value

The `evaluate_population` function iterates through each agent in the population and runs `test_model` on them to assess their fitness.

The final main function we need is the `next_generation` function in listing 6.12. Unlike our string genetic algorithm from earlier, where we probabilistically selected parents based on their fitness score, here we employ a different selection mechanism. The *probabilistic selection mechanism* is similar to how we choose actions in a policy gradient method, and it works well there, but for choosing parents in a genetic algorithm, it often ends up leading to too rapid convergence. Genetic algorithms require more exploration than gradient-descent–based methods. In this case we'll use a selection mechanism called *tournament-style selection* (figure 6.8).

Figure 6.8. In tournament selection we evaluate the fitness of all the individuals in the population as usual, and then we choose a random subset of the full population (in this figure just 2 of 4), and then choose the top individuals (usually 2) in this subset, mate them to produce offspring and mutate them. We repeat this selection process until we fill up the next generation.